



Faculdade Anhanguera superior Bacharelado em

Engenharia de Software

4º semestre

William Carlos Piccirilo Lopes

Portifólio : Relatório de aula prática

Disciplina : Fundamentos da Inteligência Artificial

Campinas

2025

William Carlos Piccirilo Lopes

Portifólio : Relatório de aula prática

Disciplina : Fundamentos da Inteligência Artificial

Aula prática de Fundamentos da Inteligência Artificial
apresentado para obtenção de média semestral no
curso de Engenharia de Software.

Tutor : Celso Soares do Nascimento

Professor: Anderson Inacio Salata de Abreu

Campinas

2025

SUMÁRIO

1	INTRODUÇÃO	3
2	DESENVOLVIMENTO	4
2.1	O que é Octave ?	4
2.2	Principais características	4
2.3	Para que é usado ?	5
3	MÉTODOS	6
3.1	Sistema de pesquisa e satisfação do cliente	6
3.2	Sistema de pesquisa e satisfação do cliente com conceitos difusos ...	7
4	DESENVOLVIMENTO REDES NEURAI	13
4.1	O que é uma rede neural ?	13
4.2	Como funciona uma rede neural ?	13
5	MÉTODOS REDE NEURAL	14
5.1	Aplicação de uma rede neural em Python	14
5.2	Treinamento da rede neural	16
6	RESULTADOS	18
7	CONCLUSÃO	19
8	REFERÊNCIAS	20

1. INTRODUÇÃO

Este relatório apresenta a atividade solicitada na disciplina de Fundamentos da Inteligência Artificial, realizada no 4º semestre do curso de Engenharia de Software.

O objetivo desta atividade foi criar uma solução para um cálculo de satisfação do cliente com o restaurante utilizando o programa Octave para as soluções de problemas numéricos e comparar o programa utilizando a Lógica Nebulosa em uma linguagem computacional.

Ainda neste relatório será mostrado como implementar uma rede neural utilizando a função sigmoide em uma linguagem em Python dentro da plataforma online Jdoodle IDE.

Neste relatório, será descrito o passo a passo para o desenvolvimento destas atividades.

2. DESENVOLVIMENTO

2.1 O que é o Octave ?

O software **GNU Octave** é uma linguagem de programação de alto nível e um ambiente para computação numérica e gráfica. É uma alternativa gratuita e de código aberto ao MATLAB, sendo amplamente utilizada em engenharia e ciência para resolver problemas matemáticos, realizar cálculos numéricos e criar gráficos.

O Octave é compatível com o MATLAB e possui uma interface de linha de comando para executar diversas tarefas e desenvolver programas.

2.2 Principais características

- **Software livre:** O Octave é gratuito e de código aberto, distribuído sob a licença GPL.
- **Compatibilidade com MATLAB:** Possui uma linguagem e uma grande quantidade de funções compatíveis com o MATLAB, tornando a transição e o uso de códigos já existentes mais fáceis.
- **Computação numérica:** É projetado para realizar cálculos numéricos, resolver problemas lineares e não lineares, e fazer experimentos numéricos.
- **Linguagem de alto nível:** Sua linguagem é considerada simples e vantajosa, especialmente para trabalhar com arrays e matrizes.

- **Multiplataforma:** Está disponível para diferentes sistemas operacionais, como Linux, macOS e Windows.
- **Funcionalidades:** Inclui recursos como editor de scripts, interface gráfica (GUI) com janela de comandos, e suporte para visualização de dados através de gráficos.

2.3 Para que é usado ?

O GNU/Octave conta com interação através de uma linguagem de programação de alto nível, programada inicialmente para cálculo numérico. Fornece uma linha de comando conveniente resolvendo problemas lineares e não-lineares numericamente, e executando outras tarefas numéricas usando uma linguagem que é bastante compatível com o MATLAB.

Pode também ser usado como uma linguagem orientada a grupos e algumas áreas específicas como:

- **Engenharia:** Usado para resolver problemas de engenharia química, como balanços de massa e energia, termodinâmica e fenômenos de transporte.
- **Educação:** Serve como ferramenta didática para cursos de cálculo numérico e outras disciplinas técnicas.
- **Pesquisa:** Utilizado em ambientes acadêmicos e de pesquisa para analisar e processar dados.

3. MÉTODOS

3.1 Sistema de pesquisa e satisfação do cliente

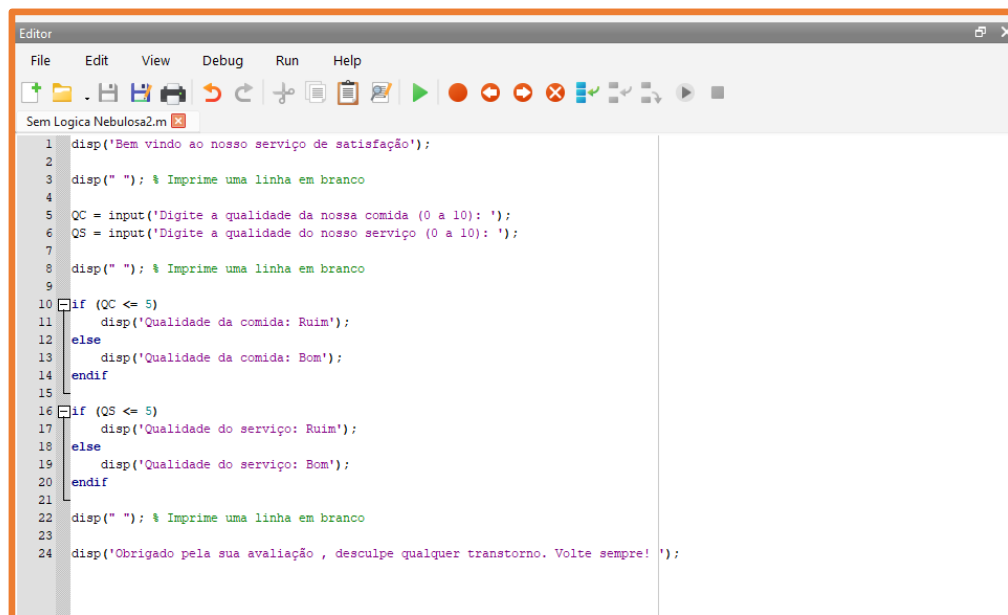
Logo abaixo podemos ver um programa de pesquisa e satisfação do cliente com restaurante. Onde ele irá definir se a comida e o serviço será **bom** ou **ruim** sem uma **lógica nebulosa**.

Primeiro atribuímos uma regra simples onde cliente avalia de 0 a 10 a comida e o serviço.

Se for :

- Ruim – nota 5 ou menor.
- Bom – nota maior que 5.

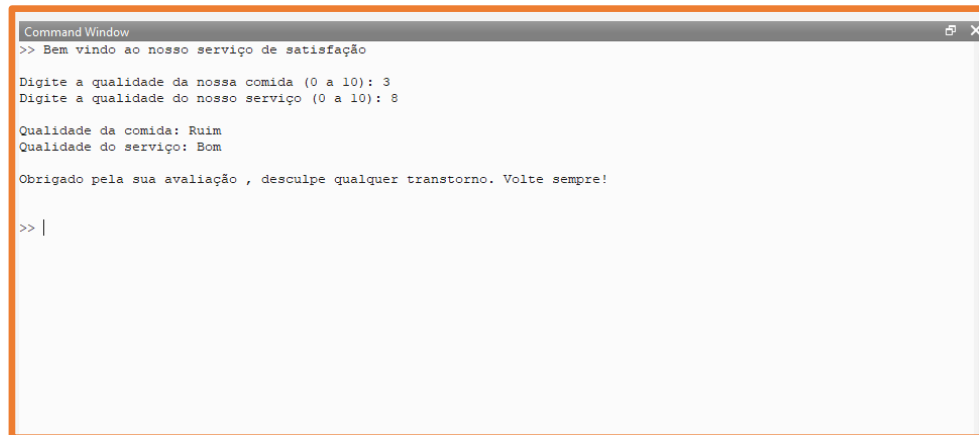
Script

The image shows a screenshot of a MATLAB script editor window titled 'Editor'. The window has a menu bar with 'File', 'Edit', 'View', 'Debug', 'Run', and 'Help'. Below the menu bar is a toolbar with various icons for file operations, editing, and running. The script file is named 'Sem Logica Nebulosa2.m'. The code is as follows:

```
1 disp('Bem vindo ao nosso serviço de satisfação');
2
3 disp(" "); % Imprime uma linha em branco
4
5 QC = input('Digite a qualidade da nossa comida (0 a 10): ');
6 QS = input('Digite a qualidade do nosso serviço (0 a 10): ');
7
8 disp(" "); % Imprime uma linha em branco
9
10 if (QC <= 5)
11     disp('Qualidade da comida: Ruim');
12 else
13     disp('Qualidade da comida: Bom');
14 endif
15
16 if (QS <= 5)
17     disp('Qualidade do serviço: Ruim');
18 else
19     disp('Qualidade do serviço: Bom');
20 endif
21
22 disp(" "); % Imprime uma linha em branco
23
24 disp('Obrigado pela sua avaliação , desculpe qualquer transtorno. Volte sempre! ');
```

Figura.1

Comando

A screenshot of a 'Command Window' interface. The text inside the window is as follows:

```
>> Bem vindo ao nosso serviço de satisfação  
  
Digite a qualidade da nossa comida (0 a 10): 3  
Digite a qualidade do nosso serviço (0 a 10): 8  
  
Qualidade da comida: Ruim  
Qualidade do serviço: Bom  
  
Obrigado pela sua avaliação , desculpe qualquer transtorno. Volte sempre!  
  
>> |
```

Figura.2

3.2 Sistema de pesquisa e satisfação do cliente com conceitos difusos.

Agora logo abaixo iremos introduzir o conceito difuso onde definimos entrada e saída quando nos deparamos com **nota** , **porcentagem** e **avaliação** entre ruim , mediano e excelente.

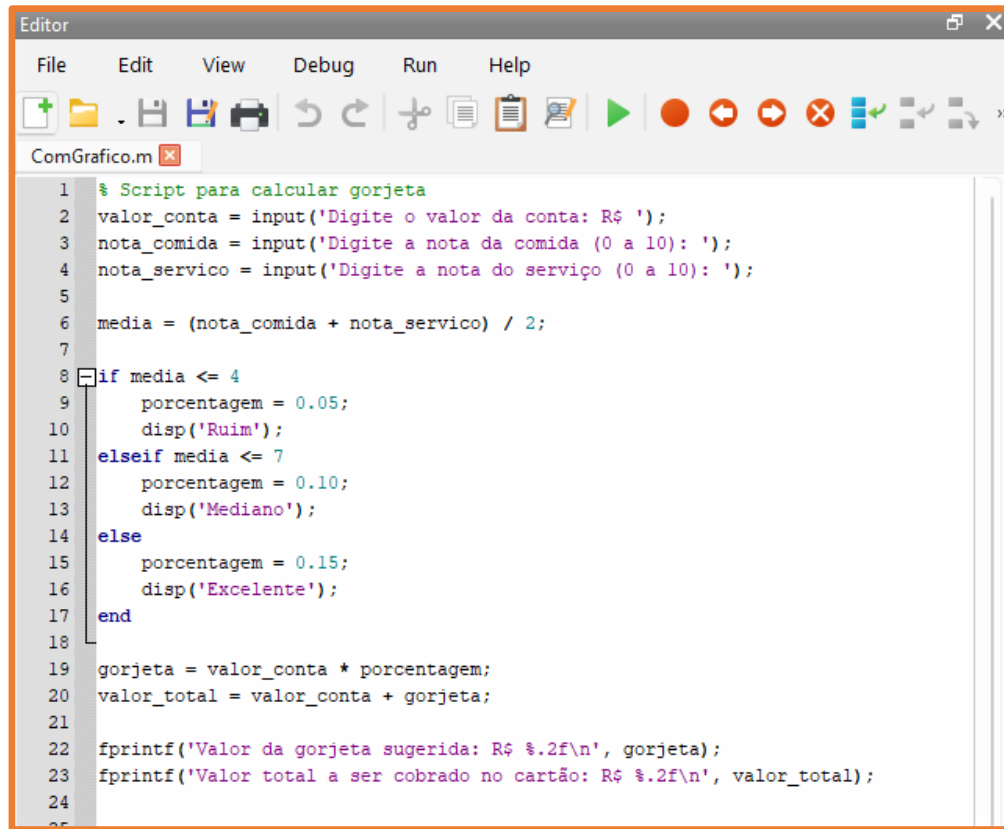
Se o conjunto fuzzy:

Qualidade da comida e serviço em nota de (0 e 10).

Qualidade da comida e serviço em porcentagem.

- Ruim - menor ou igual a 5%
- Mediano – maior que 5% ou igual a 10%
- Excelente – para maior que 10% até 15%.

Script para cálculo da gorjeta



```
1 % Script para calcular gorjeta
2 valor_conta = input('Digite o valor da conta: R$ ');
3 nota_comida = input('Digite a nota da comida (0 a 10): ');
4 nota_servico = input('Digite a nota do serviço (0 a 10): ');
5
6 media = (nota_comida + nota_servico) / 2;
7
8 if media <= 4
9     porcentagem = 0.05;
10    disp('Ruim');
11 elseif media <= 7
12    porcentagem = 0.10;
13    disp('Mediano');
14 else
15    porcentagem = 0.15;
16    disp('Excelente');
17 end
18
19 gorjeta = valor_conta * porcentagem;
20 valor_total = valor_conta + gorjeta;
21
22 fprintf('Valor da gorjeta sugerida: R$ %.2f\n', gorjeta);
23 fprintf('Valor total a ser cobrado no cartão: R$ %.2f\n', valor_total);
24
25
```

Figura.3

Script do Gráfico

```
24
25
26 %--- Gráfico 3D ---
27 [x, y] = meshgrid(0:0.5:10, 0:0.5:10);
28 media_grid = (x + y) / 2;
29
30 porcentagem_grid = zeros(size(media_grid));
31 porcentagem_grid(media_grid <= 4) = 0.05;
32 porcentagem_grid(media_grid > 4 & media_grid <= 7) = 0.10;
33 porcentagem_grid(media_grid > 7) = 0.15;
34
35 gorjeta_percent = porcentagem_grid * 100;
36
37 figure;
38 surf(x, y, gorjeta_percent);
39 xlabel('Comida');
40 ylabel('Serviço');
41 zlabel('Gorjeta (%)');
42 title('Gorjeta em função das notas');
43 colormap(jet);
44 shading interp;
45 colorbar;
46 grid on;
47 hold on;
48
49 % --- Marca o ponto da nota inserida ---
50 plot3(nota_comida, nota_servico, porcentagem * 100, 'ro', 'MarkerSize', 10, 'Line
51 text(nota_comida, nota_servico, porcentagem * 100 + 1, 'Sua Nota', 'Color', 'r',
52
53 % --- Linha até a base ---
54 plot3([nota_comida nota_comida], [nota_servico nota_servico], [0 porcentagem * 100
55
56 % --- Rotação automática ---
57 for angulo = 0:360
58     view(angulo, 30);
59     pause(0.05);
60 end
61
62
```

line: 41 | col: 23 | encoding: UTF-8 | eol: CRLF

Command Window | Documentation | Variable Editor | Editor

Figura.4

Comando com Gráfico (satisfação ruim)

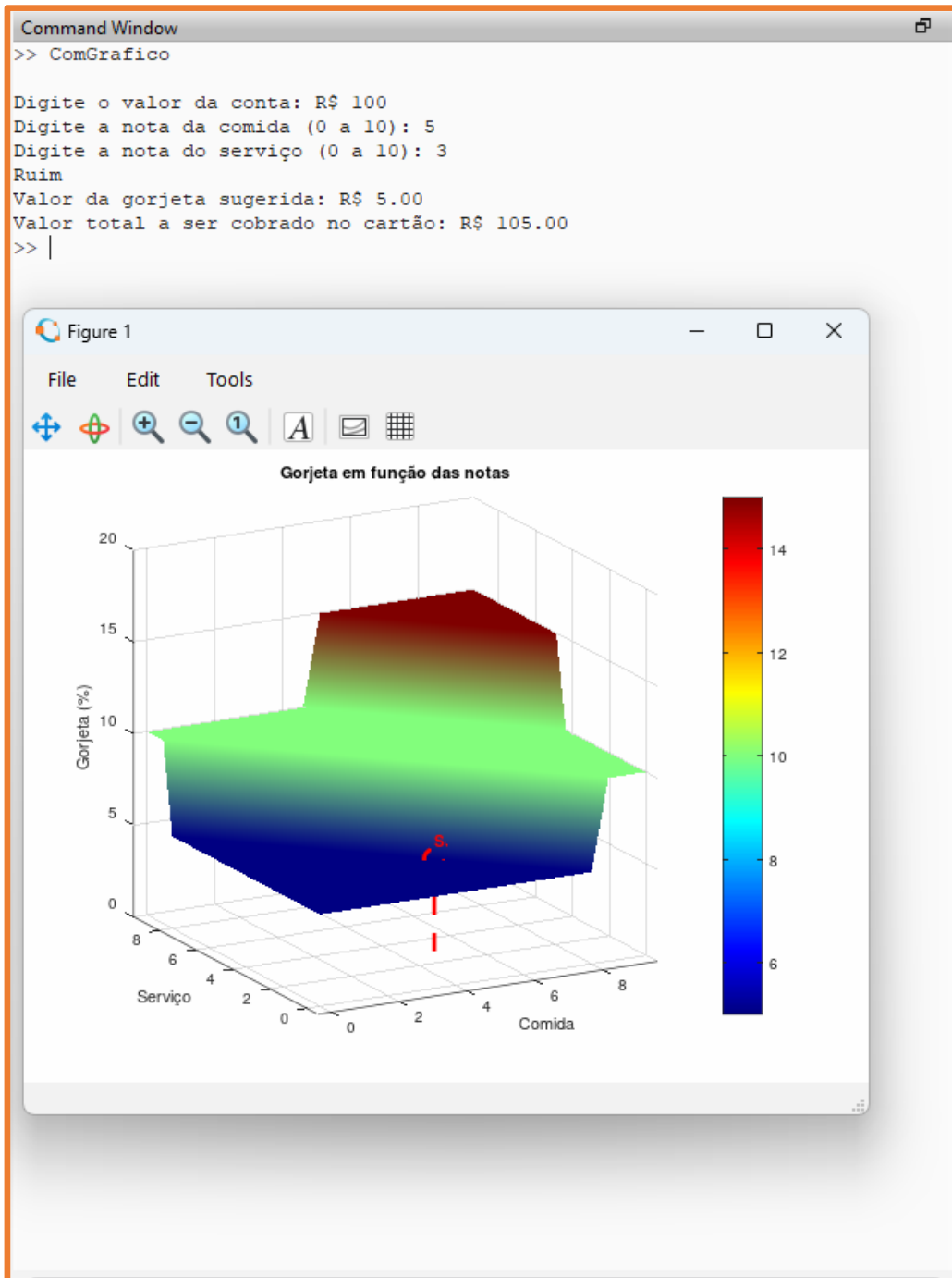


Figura.5

Comando com Gráfico (satisfação mediana)

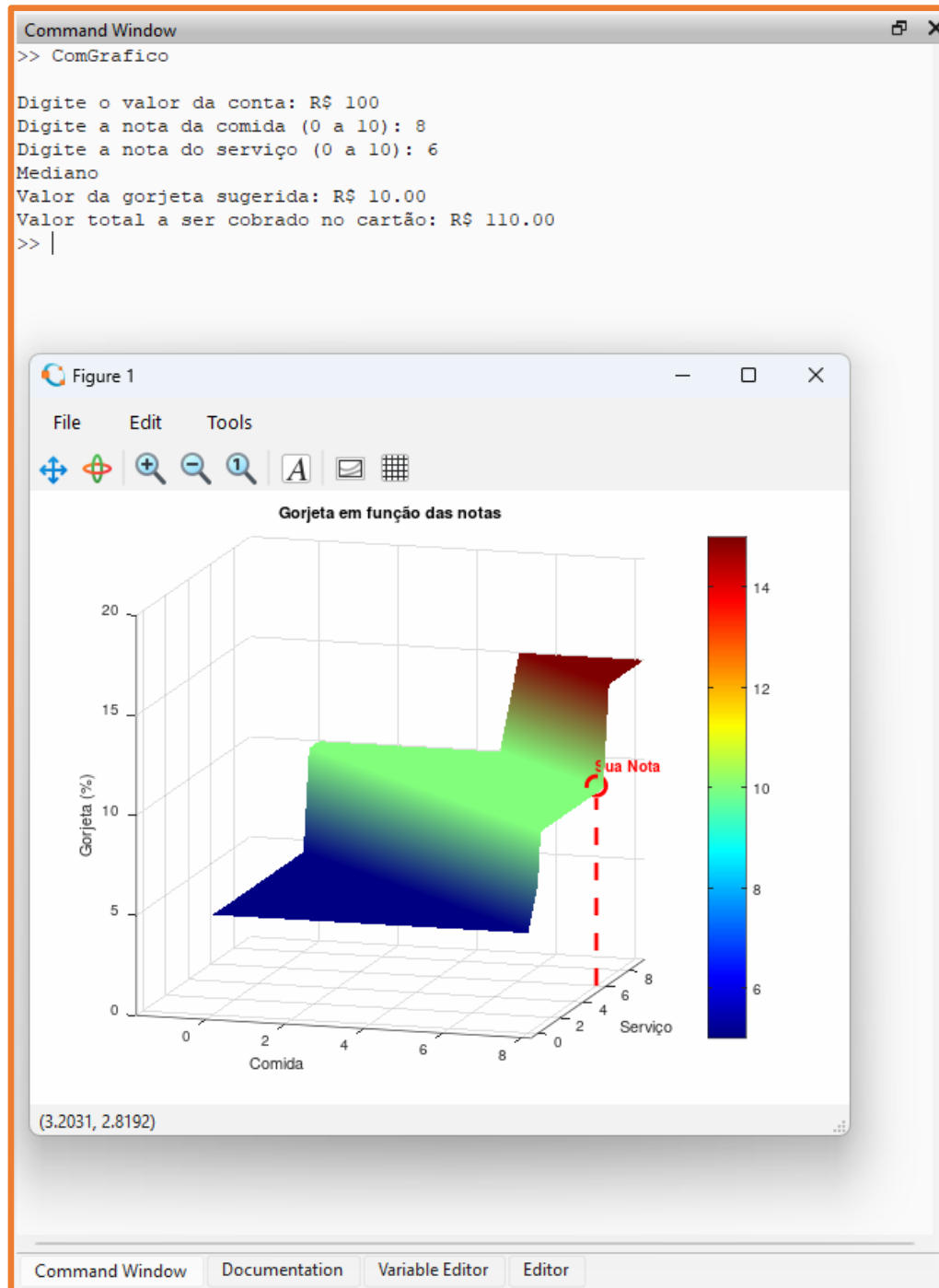


Figura.6

Comando com Gráfico (satisfação excelente)

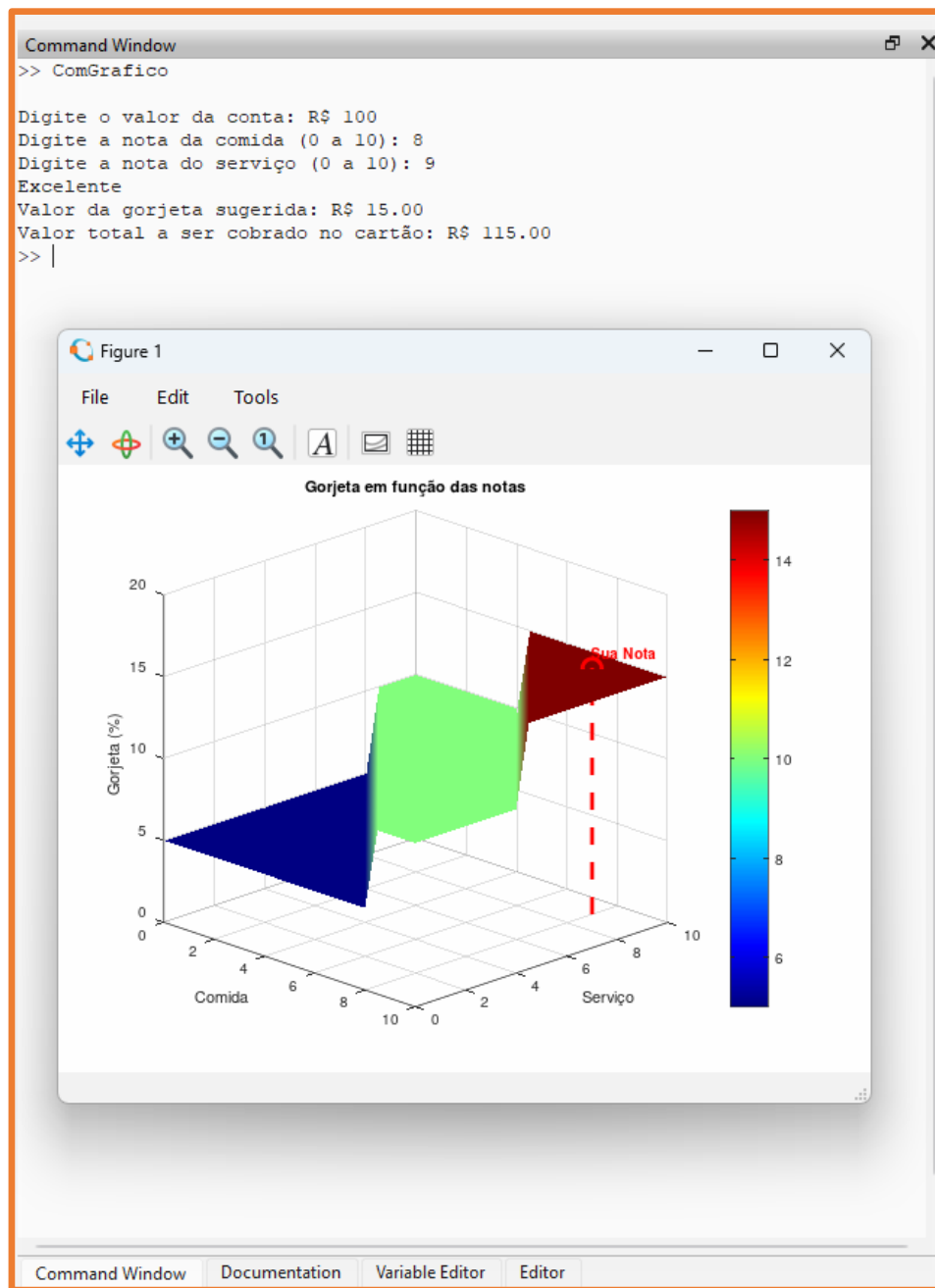


Figura.7

4. DESENVOLVIMENTO REDES NEURAIS

4.1 O que é uma rede neural?

Uma rede neural é um modelo computacional inspirado no funcionamento do cérebro humano, usado para reconhecer padrões e resolver problemas complexos.

A definição de uma rede neural é composta por **camadas de neurônios artificiais** (ou nós), que processam informações. Cada neurônio recebe dados, aplica uma função matemática e envia o resultado para outros neurônios.

4.2 Como funciona uma rede neural?

Podemos classificar a rede neural em 4 partes.

- **Entradas:** Dados entram na rede (por exemplo, números, imagens, texto).
- **Pesos e Bias:** Cada conexão tem um peso que indica a importância do dado.
- **Função de Ativação:** Decide se o neurônio “dispara” ou não (ex.: sigmoid, ReLU).
- **Saída:** A rede gera um resultado (ex.: prever se uma imagem é um gato ou cachorro).

Sua estrutura básica é composta por:

- **Camada de entrada:** Recebe os dados.
- **Camadas ocultas:** Fazem cálculos e aprendem padrões.
- **Camada de saída:** Dá a resposta final.

Assim como exemplo ilustrativo.

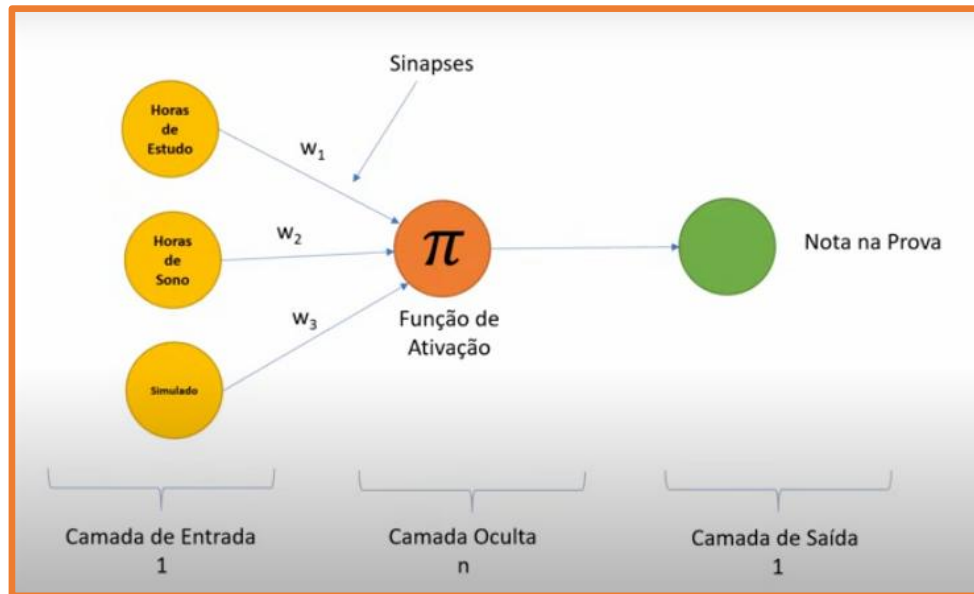


Figura.8

5. MÉTODOS REDE NEURAL

5.1 Aplicação de uma rede neural em Python

Conforme a figura (**Figura.9**) abaixo podemos ver os passos para uma rede neural de função sigmoide.

```
1 import numpy as np
2
3 # Função sigmoide e sua derivada
4 def sigmoid(x):
5     return 1 / (1 + np.exp(-x))
6
7 def sigmoid_derivative(x):
8     return x * (1 - x)
9
10 # Entradas (X) e saídas (y)
11 X = np.array([[0, 0],
12               [0, 1],
13               [1, 0],
14               [1, 1]])
15
16 y = np.array([[0],
17               [1],
18               [1],
19               [0]])
20
21 # Inicialização dos pesos
22 np.random.seed(1)
23 input_layer_neurons = X.shape[1]
24 hidden_layer_neurons = 4
25 output_neurons = 1
26
27 # Pesos e bias
28 W1 = np.random.uniform(size=(input_layer_neurons, hidden_layer_neurons))
29 b1 = np.random.uniform(size=(1, hidden_layer_neurons))
30 W2 = np.random.uniform(size=(hidden_layer_neurons, output_neurons))
31 b2 = np.random.uniform(size=(1, output_neurons))
32
33 # Treinamento
34 epochs = 10000
35 learning_rate = 0.1
36
37 for _ in range(epochs):
38     # Propagação para frente
39     hidden_input = np.dot(X, W1) + b1
40     hidden_output = sigmoid(hidden_input)
41
42     final_input = np.dot(hidden_output, W2) + b2
43     final_output = sigmoid(final_input)
44
45     # Cálculo do erro
46     error = y - final_output
47
48     # Cálculo do delta
49     d_output = error * sigmoid_derivative(final_output)
50     error_hidden = d_output.dot(W2.T)
51     d_hidden = error_hidden * sigmoid_derivative(hidden_output)
52
53     # Atualização dos pesos
54     W2 += hidden_output.T.dot(d_output) * learning_rate
55     b2 += np.sum(d_output, axis=0, keepdims=True) * learning_rate
56     W1 += X.T.dot(d_hidden) * learning_rate
57     b1 += np.sum(d_hidden, axis=0, keepdims=True) * learning_rate
```

Figura.9

Importação e funções de ativação:

- Sigmoid: Função de ativação que transforma valores em um intervalo entre 0 e 1.
- Sigmoid_derivative: Derivada da sigmoide , usada no cálculo do gradiente na retropropagação.

```
1 import numpy as np
2
3 # Função sigmoide e sua derivada
4 def sigmoid(x):
5     return 1 / (1 + np.exp(-x))
6
7 def sigmoid_derivative(x):
8     return x * (1 - x)
```

Figura.10

Dados de entrada e saída:

- X: Conjunto de entradas (XOR).
- Y: Saídas esperadas(XOR → 0 quando iguais, 1 quando diferentes).

```
10 # Entradas (X) e saídas (y)
11 X = np.array([[0, 0],
12              [0, 1],
13              [1, 0],
14              [1, 1]])
15
16 y = np.array([[0],
17              [1],
18              [1],
19              [0]])
20
```

Figura.11

Inicialização dos pesos:

- Pesos e bias são inicializados aleatoriamente.

```
21 # Inicialização dos pesos
22 np.random.seed(1)
23 input_layer_neurons = X.shape[1]
24 hidden_layer_neurons = 4
25 output_neurons = 1
26
27 # Pesos e bias
28 W1 = np.random.uniform(size=(input_layer_neurons, hidden_layer_neurons))
29 b1 = np.random.uniform(size=(1, hidden_layer_neurons))
30 W2 = np.random.uniform(size=(hidden_layer_neurons, output_neurons))
31 b2 = np.random.uniform(size=(1, output_neurons))
32
```

Figura.12

5.2 Treinamento da rede neural

Loop de treinamento propagação para frente:

- **Calcula saída da camada oculta:**

$\text{hidden_output} = \text{sigmoid}(\text{np.dot}(x, w1) + b1)$

- **Calcula saída final:**

$\text{Final output} = \text{sigmoid}(\text{np.dot}(\text{hidden output}, w2) + b2)$

```
33 # Treinamento
34 epochs = 10000
35 learning_rate = 0.1
36
37 for _ in range(epochs):
38     # Propagação para frente
39     hidden_input = np.dot(X, W1) + b1
40     hidden_output = sigmoid(hidden_input)
41
42     final_input = np.dot(hidden_output, W2) + b2
43     final_output = sigmoid(final_input)
44
```

Figura.13

Erro:

$\text{error} = y - \text{final_output}$

Retropropagação:

- **Calcula gradiente da saída:**
 $\text{d_output} = \text{error} * \text{sigmoid_derivative}(\text{final_output})$
- **Propaga erro para camada oculta:**
 $\text{d_hidden} = (\text{d_output}.\text{dot}(W2.T)) * \text{sigmoid_derivative}(\text{hidden_output})$

```
45     # Cálculo do erro
46     error = y - final_output
47
48     # Cálculo do delta
49     d_output = error * sigmoid_derivative(final_output)
50     error_hidden = d_output.dot(W2.T)
51     d_hidden = error_hidden * sigmoid_derivative(hidden_output)
52
```

Figura.14

Atualizações dos pesos:

```
53     # Atualização dos pesos
54     W2 += hidden_output.T.dot(d_output) * learning_rate
55     b2 += np.sum(d_output, axis=0, keepdims=True) * learning_rate
56     W1 += X.T.dot(d_hidden) * learning_rate
57     b1 += np.sum(d_hidden, axis=0, keepdims=True) * learning_rate
```

Figura.15

Resultado:

```
59     # Exibição dos resultados
60     print("Saída após o treinamento:")
61     print(final_output)
```

```
Saída após o treinamento:
[[0.05952174]
 [0.93776595]
 [0.9173396 ]
 [0.08658718]]
```

Figura.16

6. RESULTADOS

Ao desenvolver estas atividades pode-se perceber a facilidade na utilização do software **Octave** para criar uma solução para um problema difuso utilizando a Lógica Nebulosa para um sistema de pesquisa e satisfação do cliente com o restaurante.

O resultado foi obtido com sucesso percebendo a diferença na utilização do método com Lógica Nebulosa e sem esta técnica.

Ainda neste trabalho obtive uma ótima compreensão na utilização da plataforma **Jdoodle IDE** juntamente com a linguagem em **Python** para implementação de uma rede neural obtendo a saída necessária para o ajuste dos pesos utilizando a derivada da função sigmode.

7. CONCLUSÃO

Ao final destas atividades pode-se concluir que ao utilizar o programa **Octave**, obtive uma facilidade em aprender e desenvolver de uma forma dinâmica e versátil utilizando alguns conceitos básicos, porém, eficientes para o objetivo proposto que era criar uma solução para o sistema de pesquisa e satisfação comparando com a lógica nebulosa.

Assim também na compreensão de uma rede neural implementando em uma linguagem em Python para obter os resultados da quantidade de treinamento.

8. REFERÊNCIAS

https://pt.wikipedia.org/wiki/GNU_Octave

<https://es.mathworks.com/help/fuzzy/an-introductory-example-fuzzy-versus-nonfuzzy-logic.html>

https://pt.wikipedia.org/wiki/L%C3%B3gica_difusa

<https://cloud.google.com/discover/what-is-a-neural-network?hl=pt-BR>

<https://www.jdoodle.com/>